

UNIT-4

The dataset provided below contain the 18 days workout sessions of a person which contains, duration of workout, date, avg pulse rate and maximum pulse rate and the calories burnt.

exercise.csv

	Duration	Date	Pulse	Maxpulse	Calories
0	60	'2020/12/01'	110	130	409.1
1	60	'2020/12/02'	117	145	479.0
3	60	'2020/12/07'	110	136	374.0
4	450	'2020/12/08'	104	134	253.3
5	30	'2020/12/09'	109	133	195.1
6	60	'2020/12/10'	98	124	269.0
7	60	'2020/12/11'	103	147	329.3
8	60	'2020/12/12'	100	120	250.7
9	60	'2020/12/12'	100	120	250.7
10	45	'2020/12/18'	90	112	NaN
11	60	'2020/12/19'	103	123	323.0
12	45	'2020/12/20'	97	125	243.0
13	60	'2020/12/21'	108	131	364.2
14	45	NaN	100	119	282.0
15	60	'2020/12/25'	102	126	334.5
16	60	2020/12/26	100	120	250.0
17	60	'2020/12/27'	92	118	241.0
18	60	'2020/12/28'	103	132	NaN

Answer the programming questions below based on the dataset provided

NOTE: the first 4 lines should be present in all the answers.

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset
df = pd.read_csv('exercise_data.csv')
```

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# Load dataset
df = pd.read_csv('exercise_data.csv')
# Q1. Quick overview of the dataset
print("First 5 Records:\n", df.head())
print("\nData Types and Non-Null Counts:\n")
df.info() # no need to wrap in print()
print("\nMissing Values Per Column:\n", df.isnull().sum())

-----

# Q2. Handling missing data
print("\nShape before drop:", df.shape)
df_clean = df.dropna() # drop rows with any missing values
print("Shape after drop:", df_clean.shape)

# Fill missing values in 'Calories' column with its mean
df['Calories'].fillna(df['Calories'].mean(), inplace=True)
print("\nMissing 'Calories' filled with average")

-----
```

```
# Q3. Convert 'Date' column to datetime and add calculated field
print("\nDate column before conversion:")
print(df['Date'].head())

df['Date'] = pd.to_datetime(df['Date'], errors='coerce') # invalid dates will become NaT
print("Date column after conversion:", df['Date'].dtypes)
print("Invalid date entries:", df['Date'].isnull().sum())

# Add a new column: Calories per minute
df['calories_per_minute'] = df['Calories'] / df['Duration']
print("\nCalories per minute column added:")
print(df[['Duration', 'Calories', 'calories_per_minute']].head())

-----

# Q4. Apply basic statistics and filters
print("\nAverage Pulse:", df['Pulse'].mean())

print("\nRows where Pulse > 100:")
print(df[df['Pulse'] > 100])

print("\nCalories between 250 and 350:")
print(df[(df['Calories'] >= 250) & (df['Calories'] <= 350)])

print("\nEven Duration Records:")
print(df[df['Duration'] % 2 == 0])
```

```
-----  
# Q5. Grouping and sorting  
print("\nAverage Calories per Duration:")  
print(df.groupby('Duration')['Calories'].mean())  
print("\nSorted by Calories Descending:")  
print(df.sort_values(by='Calories', ascending=False))  
-----  
  
#Q6. Detect and remove duplicate records  
print("\nDuplicate Records:")  
print(df[df.duplicated()])  
  
df = df.drop_duplicates()  
print("Shape after dropping duplicates:", df.shape)  
-----  
  
# Q7. Visualization  
# Histogram of Calories  
plt.hist(df['Calories'], bins=10, color='skyblue', edgecolor='black')  
plt.title("Calories Distribution")  
plt.xlabel("Calories Burned")  
plt.ylabel("Frequency")  
plt.show()  
  
# Scatter plot: Duration vs Calories  
plt.scatter(df['Duration'], df['Calories'], color='green')  
plt.title("Duration vs Calories")  
plt.xlabel("Duration")  
plt.ylabel("Calories")  
plt.show()  
  
# Line plot: Pulse over Date  
df_sorted = df.sort_values(by='Date')  
plt.plot(df_sorted['Date'], df_sorted['Pulse'], marker='o')  
plt.title("Pulse Over Time")  
plt.xlabel("Date")  
plt.ylabel("Pulse")
```

```
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

Q8. How many total calories were burned for each max pulse rate? Also, list sessions with low duration.

```
df['Date'] = pd.to_datetime(df['Date'], errors='coerce')
print(df.groupby('Maxpulse')['Calories'].sum())
print(df[df['Duration'] < df['Duration'].mean()])
```

Q9. What's the total calorie burned per day? Also, show the highest calorie session each day and the correlation between pulse and calories.

```
print(df.groupby('Date')['Calories'].sum())
idx = df.groupby('Date')['Calories'].idxmax().dropna()
print(df.loc[idx])
print(df[['Pulse', 'Calories']].corr())
```

Q10. Show how many sessions were recorded per day and which 3 sessions had highest max pulse.

```
print(df['Date'].value_counts().sort_index())
print(df.nlargest(3, 'Maxpulse'))
```

Q11. Which workouts happened before 2020? Show average stats grouped by pulse level.

```
df['pulse_band'] = pd.cut(df['Pulse'], [0, 100, 110, 200], labels=['Low', 'Normal', 'High'])
print(df[df['Date'].dt.year < 2020])
print(df.groupby('pulse_band')['Calories'].describe())
```

Q12. Which session burned the most calories? Also, label each as Pass if Pulse ≥ 100 .

```
print(df.loc[df['Calories'].idxmax()])
df['Result'] = df['Pulse'].apply(lambda x: 'Pass' if x >= 100 else 'Fail')
print(df[['Pulse', 'Result']])
```

Q13. What is the average monthly calorie burn? What maxpulse occurred most often?

```
df['Month'] = df['Date'].dt.month
print(df.groupby('Month')['Calories'].mean())
print(df['Maxpulse'].value_counts().idxmax())
```

Q14. How many high-calorie sessions were there? What's the day-to-day change in calories?

```
df['Win'] = df['Calories'] > df['Calories'].median()
print("Win %:", df['Win'].mean() * 100)
df['Change'] = df['Calories'].diff()
print(df[['Date', 'Change']])
df['PulseRank'] = df['Pulse'].rank(pct=True) * 100
print(df[['Pulse', 'PulseRank']])
```

Q15. What's the average calorie burn per weekday? List top 5 most frequent maxpulse values.

```
df['Weekday'] = df['Date'].dt.day_name()
print(df.groupby('Weekday')['Calories'].mean())
print(df['Maxpulse'].value_counts().head(5))
```

Q16. How many high-calorie sessions (>400) were there each year? Also, plot sessions per day.

```
hot = df[df['Calories'] > 400]
print(hot.groupby(hot['Date'].dt.year).size())
count = df['Date'].value_counts().sort_index()
count.plot(title='Workouts Per Day', kind='line')
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

UNIT-3

1. The $2-\sqrt{2}$ can be computed as the root of the function $f(x)=x^2-2$. Starting at $a=0$ and $b=2$, use *my_bisection* to approximate the $2-\sqrt{2}$ to a tolerance of $|f(x)|<0.1$ and $|f(x)|<0.01$. Verify that the results are close to a root by plugging the root back into the function.(BISECTION)

```
import numpy as np

def my_bisection(f, a, b, tol):
    if np.sign(f(a)) == np.sign(f(b)):
        raise Exception(
            "The scalars a and b do not bound a root")
    m = (a + b) / 2
    if np.abs(f(m)) < tol:
        return m
    elif np.sign(f(a)) == np.sign(f(m)):
        return my_bisection(f, m, b, tol)
    elif np.sign(f(b)) == np.sign(f(m)):
        return my_bisection(f, a, m, tol)

f = lambda x: x ** 2 - 2
r1 = my_bisection(f, 0, 2, 0.1)
print("r1 =", r1)
r01 = my_bisection(f, 0, 2, 0.01)
print("r01 =", r01)

print("f(r1) =", f(r1))
print("f(r01) =", f(r01))
```


2. Use *my_newton*= to compute $2-\sqrt{2}$ to within tolerance of $1e-6$ starting at $x_0 = 1.5$.(NETWON-RAPHSON)

```
import numpy as np

f = lambda x: x**2 - 2
f_prime = lambda x: 2*x
newton_raphson = 1.4 - (f(1.4))/(f_prime(1.4))

print("newton_raphson =", newton_raphson)
print("sqrt(2) =", np.sqrt(2))

def my_newton(f, df, x0, tol):
    # output is an estimation of the root of f
    # using the Newton Raphson method
    # recursive implementation
    if abs(f(x0)) < tol:
        return x0
    else:
        return my_newton(f, df, x0 - f(x0)/df(x0), tol)

estimate = my_newton(f, f_prime, 1.5, 1e-6)
print("estimate =", estimate)
print("sqrt(2) =", np.sqrt(2))
```

3. Use the Trapezoid Rule to approximate $\int_0^\pi \sin(x) dx$ with 11 evenly spaced grid points over the whole interval. Compare this value to the exact value of $\int_0^\pi \sin(x) dx$.
- 2.(Trapezoid Rule)

```
import numpy as np

a = 0
b = np.pi
n = 11
h = (b - a) / (n - 1)
x = np.linspace(a, b, n)
f = np.sin(x)

I_trap = (h/2)*(f[0] + \
               2 * sum(f[1:n-1]) + f[n-1])
err_trap = 2 - I_trap

print(I_trap)
print(err_trap)
```

UNIT-2

STACK

```
# Initialize stack
stack = []

# Stack operations
def push():
    item = input("Enter item to push: ")
    stack.append(item)
    print(f"Pushed: {item}")

def pop():
    if is_empty():
        print("Stack Underflow! Cannot pop.")
    else:
        print(f"Popped: {stack.pop()}")

def peek():
    if is_empty():
        print("Stack is empty.")
    else:
        print(f"Top of stack: {stack[-1]}")

def display():
    if is_empty():
        print("Stack is empty.")
    else:
        print("Stack (Top to Bottom):", list(reversed(stack)))

def is_empty():
```

```
return len(stack) == 0

def size():
    print(f"Stack size: {len(stack)}")

# Menu loop
while True:
    print("\n--- Stack Operations Menu ---")
    print("1. Push")
    print("2. Pop")
    print("3. Peek")
    print("4. Display Stack")
    print("5. Stack Size")
    print("6. Exit")

    choice = input("Enter your choice (1-6): ")

    if choice == '1':
        push()
    elif choice == '2':
        pop()
    elif choice == '3':
        peek()
    elif choice == '4':
        display()
    elif choice == '5':
        size()
    elif choice == '6':
        print("Exiting program.")
        break
    else:
        print("Invalid choice. Please enter a number from 1 to 6.")
```

QUEUE

```
# Initialize an empty queue
queue = []

def is_empty():
    return len(queue) == 0

def enqueue(item):
    queue.append(item) # Add item at the end

def dequeue():
    if is_empty():
        return "Queue is empty"
    return queue.pop(0) # Remove item from the front

def peek():
    if is_empty():
        return "Queue is empty"
    return queue[0] # Front item

def size():
    return len(queue)

def display():
    print("Queue:", queue)

# Example usage
enqueue(10)
enqueue(20)
enqueue(30)
```

```
display()

print("Dequeued:", dequeue())
display()

print("Front element:", peek())
print("Queue size:", size())
```

LINEAR SEARCH

```
#write a program to implement linear search using recursion
def linerec(arr, key, index=0):
    if index >= len(arr):
        return -1
    if arr[index] == key:
        return index
    return linerec(arr, key, index + 1) # recursion

arr = [5, 8, 2, 9, 1]
key = 9
result = linerec(arr, key) #function call
if result != -1:
    print(f"Element found at index {result}")
else:
    print("Element not found")

PS C:\Users\masrath parveen\onedrive\desktop> python f.py
Element found at index 3
```

BINARY SEARCH

```
#Program to implement Binary Search using recursion.
```

```
def bs(arr, key, low, high):
```

```
    if low > high:
```

```
        return -1 # Base case: target not found
```

```
    mid = (low + high) // 2
```

```
    if arr[mid] == key:
```

```
        return mid # Target found
```

```
    elif arr[mid] < key:
```

```
        return bs(arr, key, mid + 1, high) # Search right half
```

```
    else:
```

```
        return bs(arr, key, low, mid - 1) # Search left half
```

```
arr = [1,3,5,7,9,11,13]
```

```
key = 2
```

```
result = bs(arr, key,0,len(arr)-1)
```

```
if result != -1:
```

```
    print(f"the element {key} found at index {result}")
```

```
else:
```

```
    print(f" the element {key} not found")
```

```
PS C:\Users\masrath parveen\onedrive\desktop> python f.py
```

```
the element 13 found at index 6
```

BUBBLE SORT

```
#write Program / function to implement Bubble Sort
def bubble(arr):
    n = len(arr)
    for i in range(n):
        swapped = False # Flag to detect any swap
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                # Swap if elements are in the wrong order
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        # If no two elements were swapped by inner loop, then break
        if not swapped:
            break

arr = [64, 34, 25, 12, 22, 11, 90]
print("Original Array:", arr)

bubble(arr)
print("Sorted Array:", arr)
```

PS C:\Users\masrath parveen\onedrive\desktop> python f.py

Original Array: [64, 34, 25, 12, 22, 11, 90]

Sorted Array: [11, 12, 22, 25, 34, 64, 90]

How do you create a list in Python and perform the following operations?

Create a list

Add elements to the list

Remove elements from the list

Update elements in the list

```
my_list = [10, 20, 30]
print("Initial List:", my_list)
# Adding elements
my_list.append(40)
my_list.extend([50, 60])
my_list.insert(2, 25)
print("After Additions:", my_list)
# Removing elements
my_list.remove(30)
popped = my_list.pop()
print("Popped Element:", popped)
my_list.pop(1)
del my_list[0]
print("After Removals:", my_list)
# Updating elements
my_list[1] = 99
print("After Update:", my_list)
# Clearing all elements
my_list.clear()
print("After Clear:", my_list)
```

Write a Python program to create a tuple of five numbers. Then:

- 1. Print the first and last elements.**
- 2. Find and print the length of the tuple.**
- 3. Try changing the second element and observe what happens.**

```
# Create a tuple of numbers
numbers = (10, 20, 30, 40, 50)

# Print first and last elements
print("First element:", numbers[0])
print("Last element:", numbers[-1])

# Print length of the tuple
print("Length of tuple:", len(numbers))

# Try changing the second element (this will raise an error)
numbers[1] = 25 # ✗ Tuples are immutable, so this will cause a TypeError
```

Write a Python program to create a set of fruits. Add a new fruit to the set, remove one fruit, and then check if a specific fruit (e.g., "Apple") is present in the set.

```
# Create a set of fruits
fruits = {"Apple", "Banana", "Mango"}

# Add a new fruit
fruits.add("Orange")

# Remove one fruit
fruits.remove("Banana") # Make sure "Banana" exists

# Check for membership
if "Apple" in fruits:
    print("Apple is in the set.")
else:
    print("Apple is not in the set.")

# Print final set
print("Current set of fruits:", fruits)
```

Write a Python program to create a dictionary storing names of students as keys and their marks as values. Add a new student, update an existing student's marks, and print all student names with their marks.

```
# Create dictionary of students and their marks
student_marks = {
    "A": 85,
    "B": 90,
    "C": 78
}

# Add a new student
student_marks["D"] = 92

# Update marks for an existing student
student_marks["B"] = 95

# Print all students and their marks
print("Student Marks:")
for name, marks in student_marks.items():
    print(f'{name}: {marks}')
```

UNIT-1

Write a Python program to find the factorial of a given number.

```
# Input from user
num = int(input("Enter a number: "))

# Initialize factorial result
factorial = 1

# Check if the number is negative
if num < 0:
    print("Factorial is not defined for negative numbers.")
elif num == 0 or num == 1:
    print("Factorial of", num, "is 1")
else:
    for i in range(2, num + 1):
        factorial *= i
    print("Factorial of", num, "is", factorial)
```

Write a Python program to print the Fibonacci sequence up to n terms.

```
# Input: number of terms
n = int(input("Enter the number of terms: "))

# First two Fibonacci numbers
a, b = 0, 1

# Print Fibonacci sequence
print("Fibonacci sequence:")
for i in range(n):
    print(a, end=' ')
    a, b = b, a + b
```

Write a Python program to input three numbers and print them in ascending order without using the .sort() method.

```
# Input 3 numbers from the user
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
c = int(input("Enter third number: "))

# Compare and reorder using conditional logic
if a > b:
    a, b = b, a # swap a and b
if a > c:
    a, c = c, a # swap a and c
if b > c:
    b, c = c, b # swap b and c

# Print in ascending order
print("Numbers in ascending order:", a, b, c)
```